Efficient implementation of a Monte Carlo method for uncertainty evaluation in dynamic measurements

S Eichstädt[1], A Link[1], P Harris[2] and C Elster[1]

[1]Physikalisch-Technische Bundesanstalt, Abbestrasse 2-12, 10587 Berlin, Germany
[2]National Physical Laboratory, Teddington, Middlesex TW11 0LW, UK

E-mail: `sascha.eichstaedt@ptb.de`

Abstract. Measurement of quantities having time-dependent values such as force, acceleration or pressure is a topic of growing importance in metrology. The application of the *Guide to the Expression of Uncertainty in Measurement* (GUM) and its Supplements to the evaluation of uncertainty for such quantities is challenging. We address the efficient implementation of the Monte Carlo method described in GUM Supplements 1 and 2 for this task. The starting point is a time-domain observation equation. The steps of deriving a corresponding measurement model, the assignment of probability distributions to the input quantities in the model, and the propagation of the distributions through the model are all considered. A direct implementation of a Monte Carlo method can be intractable on many computers since the storage requirement of the method can be large compared with the available computer memory. Two memory-efficient alternatives to the direct implementation are proposed. One approach is based on applying updating formulae for calculating means, variances and point-wise histograms. The second approach is based on evaluating the measurement model sequentially in time. A simulated example is used to compare the performance of the direct and alternative procedures.

Submitted to *Metrologia*

1. Introduction

The treatment of dynamic measurements is a topic of growing importance in metrology. Examples are the measurement of acceleration and force [1, 2, 3] and calibrations undertaken using high-speed electronic devices [4, 5, 6]. The goal in the analysis of such measurements is the estimation of the time-dependent value of the measurand from limited knowledge about the measuring system and an observed output signal from the system.

One topic for research in this area is the evaluation of measurement uncertainty [7, 8, 9, 10]. The *Guide to the Expression of Uncertainty in Measurement* (GUM) [11] and its supplements, Supplement 1 (GUM–S1) and Supplement 2 (GUM–S2) [12, 13], provide a harmonized framework for the evaluation of measurement uncertainty that is a prerequisite for internationally comparable measurements. However, application of these documents to dynamic measurements is challenging. The reasons are that the derivation of the measurement model, which is the starting point for the application of the documents, can be difficult and the quantities involved in the model are high-dimensional.

The measurement model for a dynamic measurement is generally non-linear in the input quantities. Often the measurement model describes the application of a digital filter whose input is the observed output signal from the measuring system and whose output is an estimate of the time-dependent measurand [7, 8, 9, 10]. The parameters of such a filter are derived from limited knowledge about the measuring system and are, hence, uncertain [14, 15]. For models describing finite-length impulse response (FIR) filters, closed formulae for the calculation of the estimates and their associated standard uncertainties are available [16]. For models describing infinite-length impulse response (IIR) filters, an implementation of the law of propagation of uncertainty as described in the GUM has recently been presented for the evaluation of measurement uncertainty [7]. However, the magnitudes of the resulting linearization errors can be appreciable and a Monte Carlo method may be required in order to obtain reliable results. Moreover, the calculation of coverage intervals at each time instant (or point-wise) generally requires knowledge of the (point-wise) marginal probability distributions for the measurand.

The application of the propagation of distributions as described in GUM–S1 and GUM–S2 is therefore proposed. We focus here on the use of a Monte Carlo method for the calculation of means, variances and point-wise coverage intervals in the context of a dynamic measurement problem. In order to apply the propagation of distributions, we address the derivation of the measurement model, the assignment of a joint probability density function (PDF) to the input quantities in the model, and the propagation of the PDF through the model.

A direct implementation of a Monte Carlo method for the propagation of distributions consists of making a sufficient number of random draws from the joint PDF for the input quantities, evaluating the measurement model for each random draw, and calculating the required results from the model values. For long signal sequences, which

might contain thousands of values, the approach can become intractable because the storage required is large compared with the available computer memory, particularly if the elements of matrices are to be stored contiguously in memory as is done by many computer programs. We therefore present two memory-efficient alternatives. One approach is based on applying *updating formulae* for calculating means, variances and point-wise histograms. The formulae for the means and variances give results that reproduce those obtained from the direct implementation, whilst the coverage intervals derived from the point-wise histograms are approximate. To obtain point-wise coverage intervals that reproduce exactly those from the direct implementation we propose a second memory-efficient approach based on evaluating the measurement model *sequentially* in time. The approach also provides values for the means and variances and all other point-wise statistics that reproduce those obtained from the direct implementation. The sequential approach allows for an efficient implementation of GUM–S1 and GUM–S2 on standard desktop computers.

The drawback of the sequential approach is that it provides only short-term statistics and marginal PDFs, e.g., only short-term correlations are calculated. Often the estimation of the dynamic measurand is followed by an application of a functional, e.g., to evaluate an integral of the measurand. In this case the evaluation of correlations associated with estimates of the measurand at different time instants is important for the evaluation of the uncertainty associated with an estimate of the functional. We show that for many functionals the associated uncertainty evaluation can be incorporated easily into the developed sequential algorithms, and that correlations are then fully taken into account.

The paper is organized as follows. In Section 2 a problem specification and the notation used are introduced. In Section 3 we address the derivation of the measurement model and the assignment of a joint PDF to its input quantities. In Section 4 we describe the direct implementation of a Monte Carlo method for measurement uncertainty evaluation and the alternative memory-efficient procedures. Algorithmic descriptions are included for these procedures. In Section 5 we illustrate the application of the procedures by means of a simulated example. Conclusions are given in Section 6.

2. Problem specification and assumptions

We assume the measuring system is linear and time-invariant and that the (sampled) system output $\boldsymbol{X} = (X[1], \ldots, X[N])$ is modelled by the *observation equation*

$$x[n] = X[n] + \varepsilon[n] = (h_{\boldsymbol{\Gamma}} * \boldsymbol{Y})[n] + \varepsilon[n], \qquad n = 1, \ldots, N, \tag{1}$$

where $x[n]$ is an observation of $X[n]$, $\varepsilon[n]$ a zero-mean stationary process (described below), $\boldsymbol{Y} = (Y[1], \ldots, Y[N])$ the dynamic measurand, and $h_{\boldsymbol{\Gamma}}$ the impulse response of the measuring system. In equation (1) the symbol '$*$' denotes convolution of the (discrete-time) system impulse response and the (discrete-time) measurand

$$(h_{\boldsymbol{\Gamma}} * \boldsymbol{Y})[n] = \sum_j h_{\boldsymbol{\Gamma}}[j] Y[n-j], \tag{2}$$

which, for systems with an impulse response of infinite duration, is usually written in a recursive difference form [17]; see equation (4) below. The impulse response is defined by the vector $\boldsymbol{\Gamma} = (\Gamma_1, \ldots, \Gamma_D)$ containing the parameters of the discrete transfer function model, discrete values of the frequency response function or (finite) discrete impulse response function. Throughout this paper, univariate quantities, such as $X[n]$ and $Y[n]$ are written in italic and denote scalars, and multivariate quantities, such as $\boldsymbol{X}$ and $\boldsymbol{Y}$, are written in bold and denote *row vectors*.

Derivation of a measurement model for the dynamic measurand $\boldsymbol{Y}$ requires a deconvolution [18]. The deconvolution can be carried out in the frequency domain using the discrete Fourier transform [6], or in the time domain using digital filtering [10, 16, 19]. Here we consider that a digital deconvolution filter $g_{\boldsymbol{M}}(z)$ is applied to the system output $\boldsymbol{X}$. Since deconvolution is an ill-posed problem [18], regularization is applied. Such regularization can introduce a systematic error in the estimation [14] and, hence, the measurement model is generally given by

$$Y[n] = \widetilde{Y}[n] + \Delta[n], \tag{3}$$

$$\widetilde{Y}[n] = \sum_{j=0}^{\min(N_b, n-1)} B_j X[n-j] - \sum_{j=1}^{\min(N_a, n-1)} A_j \widetilde{Y}[n-j], \tag{4}$$

where $\boldsymbol{M} = (A_1, \ldots, A_{N_a}, B_0, \ldots, B_{N_b})$ are the coefficients defining $g_{\boldsymbol{M}}(z)$, and $\Delta[n]$ denotes a systematic correction to account for the systematic error introduced by the application of the deconvolution filter.

Throughout the paper the following assumptions are made:

(A1) The observational noise process $\varepsilon[n]$ in equation (1) can be modelled as a stationary zero-mean auto-regressive moving average ARMA(p, q) stochastic process:

$$\varepsilon[n] = \sum_{j=1}^{\min(p, n-1)} \varphi_j \varepsilon[n-j] + \sum_{j=1}^{\min(q, n-1)} \vartheta_j w[n-j] + w[n], \tag{5}$$

with parameters $\varphi_1, \ldots, \varphi_p, \vartheta_1, \ldots, \vartheta_q$ and zero-mean Gaussian noise process $w[n] \sim \mathcal{N}(0, \sigma^2)$ with variance σ^2. The model (5) includes as special cases independent and identically distributed Gaussian noise ($p = q = 0$) as well as discrete Gauss-Markov processes ($p = 1$, $q = 0$).

(A2) A PDF $p_{\boldsymbol{\Gamma}}(\gamma)$ for the measuring system parameters $\boldsymbol{\Gamma}$ is available from previous calibration measurements, describing the state of knowledge about these parameters.

For the identification of the ARMA process parameters, standard software tools can be used, such as the System Identification toolbox in MATLAB®. For example, the parameters φ_k and ϑ_k in model (5) can be estimated from an observed realization of the noise process $\varepsilon[n]$ by application of the MATLAB routine *armax*. For a theoretical overview we refer the reader to [20]. When the time series for the estimation of the ARMA process parameters in model (5) is sufficiently long, the uncertainty associated with the obtained estimates for the parameter vectors $\boldsymbol{\varphi}$ and $\boldsymbol{\vartheta}$ may be neglected.

3. Assignment of PDFs to the input quantities

The input quantities in the measurement model (4) and (3) are the coefficients $\boldsymbol{M} \in \mathbb{R}^{N_a+N_b+1}$ of the filter $g_{\boldsymbol{M}}$, the system output $\boldsymbol{X} \in \mathbb{R}^N$ and the systematic correction $\boldsymbol{\Delta} \in \mathbb{R}^N$. Knowledge about the input quantities is assumed to be obtained independently. Hence, the joint PDF characterizing the input quantities is the product of the individual joint PDFs characterizing $\boldsymbol{M}$, $\boldsymbol{X}$ and $\boldsymbol{\Delta}$.

The digital filter $g_{\boldsymbol{M}}(z)$ is typically designed as an approximation $g_{\text{inv}}(z)$ to the reciprocal (or inverse) of the impulse response $h_{\boldsymbol{\Gamma}}$ of the measuring system in cascade with a low pass filter $g_{\text{low}}(z)$ for noise attenuation (cf. [10, 14]) determined by the chosen cut-off frequency and filter type. The approximation $g_{\text{inv}}(z)$ to the reciprocal impulse response is obtained from limited knowledge about the measuring system. An estimate $\boldsymbol{\mu}$ of the filter coefficients $\boldsymbol{M}$ of the filter $g_{\boldsymbol{M}} = g_{\text{inv}} g_{\text{low}}$ is calculated from an estimate $\boldsymbol{\Gamma}$ of the measuring system parameters $\boldsymbol{\Gamma}$ using a mapping $\boldsymbol{M} = \boldsymbol{f}(\boldsymbol{\Gamma})$, where this mapping results from, e.g., a least-squares optimization [14, 16], an adjustment of a discrete transfer function model [10, 14], or a simple calculation of the reciprocal transfer function model. A typical example is a least-squares fit to the reciprocal of the frequency response obtained in a calibration measurement. The PDF $p_{\boldsymbol{M}}(\boldsymbol{\mu})$ for $\boldsymbol{M}$ is then obtained by applying GUM–S2 to propagate the PDF for $\boldsymbol{\Gamma}$ (cf. assumption (A2)) through the filter design algorithm.

Knowledge about the system outputs $X[n]$ is assumed to be available in the form of discrete-time observations $x[n]$ according to observation equation (1) with observational noise process $\varepsilon[n]$ described by model (5). For the assignment of PDFs we employ Bayes theorem with non-informative priors resulting in the joint posterior probability distribution $\mathcal{N}(\boldsymbol{x}, V_\varepsilon)$ for $\boldsymbol{X}$, a multivariate Gaussian distribution with expectation $\boldsymbol{x}$ and covariance matrix V_ε. The covariance matrix V_ε, with elements $(V_\varepsilon)_{k,l} = \rho(|k-l|)$, is calculated from the known auto-covariance function $\rho(m)$ of the noise process $\varepsilon[n]$. For the calculation of the auto-covariance function $\rho(m)$ of an ARMA process with known parameters see [21, 22].

The systematic correction $\Delta[n]$ in expression (3) addressing the regularization error is determined by the filter design mapping $\boldsymbol{f}(\boldsymbol{\Gamma})$ and the properties of the measurand $\boldsymbol{Y}$. Consequently, the assignment of a joint PDF $p_{\boldsymbol{\Delta}}(\boldsymbol{\delta})$ requires some prior knowledge about $\boldsymbol{Y}$. One possibility is that an upper bound $|\Delta[n]| \leq \overline{\Delta}$, independent of n, is derived from an upper bound for the absolute value of the Fourier transform of $\boldsymbol{Y}$, cf. [9, 15]. Then, following GUM–S1, $p_{\boldsymbol{\Delta}}(\boldsymbol{\delta})$ is assigned to be the probability density of the product of N probability distributions $\Delta[n] \sim \mathcal{U}(-\overline{\Delta}, \overline{\Delta})$, each a uniform distribution with lower limit $-\overline{\Delta}$ and upper limit $\overline{\Delta}$. Hereafter, for ease of presentation, we employ this type of prior knowledge. However, the presented sequential algorithms also apply to other PDFs $p_{\boldsymbol{\Delta}}(\boldsymbol{\delta})$, provided the assigned PDF allows random draws $\delta^{(k)}[n]$ to be made sequentially in time.

The resulting joint probability distribution for the input quantities $\boldsymbol{M}$, $\boldsymbol{X}$ and $\boldsymbol{\Delta}$

is then given as

$$(\boldsymbol{M}, \boldsymbol{X}, \boldsymbol{\Delta}) \sim F_{\boldsymbol{M}} \mathcal{N}(\boldsymbol{x}, V_\varepsilon) \prod_{n=1}^{N} \mathcal{U}(-\overline{\Delta}, \overline{\Delta}), \tag{6}$$

where $F_{\boldsymbol{M}}$ denotes the probability distribution for M. Making random draws from the joint PDF for the input quantities is carried out by making random draws from the individual PDFs, for which guidance is given in [12].

4. Implementation of propagation of PDFs

Application of the propagation of distributions to measurement model (4) and (3) can be carried out by considering the input quantities $\boldsymbol{M}$, $\boldsymbol{X}$ and $\boldsymbol{\Delta}$ and output quantity $\boldsymbol{Y}$ as multivariate (vector-valued) quantities. In the resulting Monte Carlo procedure random draws are made from the joint PDF (6) for the input quantities. As for the case of a univariate output quantity, the number K of Monte Carlo trials is chosen such that a prescribed numerical accuracy in the results, e.g., mean, standard uncertainty or any other characteristic, is achieved with a prescribed probability; cf. [12, 23]. For a multivariate output quantity $\boldsymbol{Y}$, as a summary criterion for the output quantity one can consider either a function of the mean vector or the individual mean vector and covariance matrix elements to monitor stabilization of the Monte Carlo results. However, for a multivariate output quantity the maximum number of Monte Carlo trials, and thus the achievable accuracy, is often limited by the available computer memory. Especially for a direct implementation of a Monte Carlo method for model (4) and (3), the maximum number of Monte Carlo trials may be small for reasonably long signal sequences. The reason is that all realizations (with $K \approx 10^6$, say) of the whole sequence $\boldsymbol{Y} = (Y[1], \ldots, Y[N])$ have to be stored, and when these sequences are long (with $N = 10^3$, say), the resulting memory requirement of the method becomes appreciable, particularly if the elements of matrices are to be stored contiguously.

4.1. Direct implementation

A direct implementation of the GUM–S2 Monte Carlo method is carried out by making random draws from the joint PDF (6) and propagating these draws through measurement model (4) and (3) to obtain random draws from the joint PDF $p_{\boldsymbol{Y}}(\boldsymbol{y})$ for the measurand $\boldsymbol{Y} = (Y[1], \ldots, Y[N])$. Hence, the result of each Monte Carlo trial is one 'batch' $\boldsymbol{y}^{(k)} = (y^{(k)}[1], \ldots, y^{(k)}[N])$ of output quantity values. We call this method 'Batch Monte Carlo' (BMC) and give an algorithmic description in Algorithm 1, where $()^\top$ in line 9 denotes vector transpose and *CalcCred* in line 11 denotes calculation of point-wise coverage intervals $\boldsymbol{I}_{Y[n]} \equiv [y_{\text{low}}[n], y_{\text{high}}[n]]$ to achieve some prescribed coverage probability P (e.g. 95 %):

$$P = \int_{y_{\text{low}}[n]}^{y_{\text{high}}[n]} p_{Y[n]}(\eta)\, \mathrm{d}\eta.$$

The outputs of the algorithm are the mean vector $\widehat{\boldsymbol{y}}$, covariance matrix $U_{\widehat{\boldsymbol{y}}}$ and coverage intervals $\boldsymbol{I}_{Y[n]}$ calculated from the set of batches $\{\boldsymbol{y}^{(1)}, \ldots, \boldsymbol{y}^{(K)}\}$.

Algorithm 1: Batch Monte Carlo (BMC)

Initialize: set K as the number of Monte Carlo trials;
for $k = 1$ **to** K **do**
  draw $\boldsymbol{x}^{(k)}$ from $\mathcal{N}(\boldsymbol{x}, V_\varepsilon)$;
  draw $\boldsymbol{\mu}^{(k)} = (a_1^{(k)}, \ldots, a_{N_a}^{(k)}, b_0^{(k)}, \ldots, b_{N_b}^{(k)})$ from $p_{\boldsymbol{M}}(\boldsymbol{\mu})$;
  draw $\boldsymbol{\delta}^{(k)}$ from $\prod_{n=1}^{N} \mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
  $\boldsymbol{y}^{(k)} = (g_{\boldsymbol{\mu}^{(k)}} * \boldsymbol{x}^{(k)}) + \boldsymbol{\delta}^{(k)}$;
end
$\widehat{\boldsymbol{y}} = \frac{1}{K} \sum_{k=1}^{K} \boldsymbol{y}^{(k)}$;
$\boldsymbol{U}_{\widehat{\boldsymbol{y}}} = \frac{1}{K-1} \sum_{k=1}^{K} (\boldsymbol{y}^{(k)} - \widehat{\boldsymbol{y}})^\top (\boldsymbol{y}^{(k)} - \widehat{\boldsymbol{y}})$;
for $n = 1$ **to** N **do**
  $\boldsymbol{I}_{Y[n]} = \texttt{CalcCred}(\{\boldsymbol{y}^{(k)}[n],\ k = 1, \ldots, K\})$;
end

In general, making random draws from $\mathcal{N}(\boldsymbol{x}, V_\varepsilon)$ is computationally expensive, as it requires the calculation of the Cholesky factorization of the covariance matrix V_ε, which requires of the order of N^3 floating-point operations. However, the assumed ARMA model (5) for the noise process $\varepsilon[n]$ allows for an efficient sequential sampling of the noise process. An algorithmic description for the sequential generation of noise process realizations $\varepsilon^{(k)}[n]$ is given in Algorithm 2. A random draw $x^{(k)}[n]$ is then generated as $x^{(k)}[n] = x[n] + \varepsilon^{(k)}[n]$.

Algorithm 2: Sequential generation of noise process realizations (SGNP)

Initialize: set K as the number of Monte Carlo trials;
for $k = 1$ **to** K **do**
  draw $w^{(k)}[1]$ from $\mathcal{N}(0, \sigma^2)$;
  $\varepsilon^{(k)}[1] = w^{(k)}[1]$;
  for $n = 2$ **to** N **do**
    draw $w^{(k)}[n]$ from $\mathcal{N}(0, \sigma^2)$;
    $\varepsilon^{(k)}[n] = \sum_{j=1}^{\min(p,n-1)} \varphi_j \varepsilon^{(k)}[n-j] + \sum_{j=1}^{\min(q,n-1)} \vartheta_j w^{(k)}[n-j] + w^{(k)}[n]$;
  end
end

4.2. Updating formulae

Updating formulae are used to avoid the need to store all K batches of output quantity values as in the BMC procedure. The approach is based on expressing the results, such as means, covariances, etc., for $K_0 + K_{\text{seq}}$ Monte Carlo trials in terms of the results for K_0 trials and K_{seq} additional batches of output quantity values. The procedure is applied iteratively starting with $K_0 = 0$ and K_{seq} chosen to be a factor of K, the total number of trials.

Suppose K_0 Monte Carlo trials have been undertaken, and the corresponding

batches of the output quantity $\boldsymbol{Y}$ are summarized by the mean vector $\widehat{\boldsymbol{y}}_0$ and covariance matrix $\boldsymbol{U}_{\widehat{\boldsymbol{y}}_0}$. Suppose a further K_{seq} Monte Carlo trials are undertaken giving batches $\boldsymbol{y}^{(k)},\ k = 1, \ldots, K_{\mathrm{seq}}$. Then, the mean vector of the combined set of $K_0 + K_{\mathrm{seq}}$ batches is

$$\widehat{\boldsymbol{y}} = \widehat{\boldsymbol{y}}_0 + \frac{1}{K_0 + K_{\mathrm{seq}}} \sum_{k=1}^{K_{\mathrm{seq}}} \left(\boldsymbol{y}^{(k)} - \widehat{\boldsymbol{y}}_0\right), \tag{7}$$

and the covariance matrix is

$$\boldsymbol{U}_{\widehat{\boldsymbol{y}}} = \frac{1}{K_0 + K_{\mathrm{seq}} - 1} \Big[(K_0 - 1)\boldsymbol{U}_{\widehat{\boldsymbol{y}}_0} + K_0(\widehat{\boldsymbol{y}} - \widehat{\boldsymbol{y}}_0)^\top(\widehat{\boldsymbol{y}} - \widehat{\boldsymbol{y}}_0) + \sum_{k=1}^{K_{\mathrm{seq}}}(\boldsymbol{y}^{(k)} - \widehat{\boldsymbol{y}})^\top(\boldsymbol{y}^{(k)} - \widehat{\boldsymbol{y}})\Big]. \tag{8}$$

For each time instant n, an approximate coverage interval for $Y[n]$ can be determined as follows. Let $-\infty = \eta_0[n] < \eta_1[n] < \cdots < \eta_{b-1}[n] < \eta_b[n] = \infty$ define a partition of $\mathbb{R}$ composed of the d intervals

$$I_1[n] = (\eta_0[n], \eta_1[n]), \quad I_j[n] = [\eta_{j-1}[n], \eta_j[n]),\ j = 2, \ldots, d.$$

The output quantity values from an initial number, 1 000 say, of Monte Carlo trials can be used to decide suitable values for $\eta_1[n], \ldots, \eta_{d-1}[n]$, which are expected to be different for each n. Let $f_{0,j}[n],\ j = 1, \ldots, d$, be the number of output quantity values from K_0 trials contained in the interval $I_j[n]$. After an additional K_{seq} trials, the number $f_j[n]$ of values in $I_j[n]$ is

$$f_j[n] = f_{0,j}[n] + \mathrm{card}(\{y^{(k)}[n] \in I_j[n] : k = 1, \ldots, K_{\mathrm{seq}}\}), \tag{9}$$

where 'card(A)' is used to denote the cardinality of the set A, i.e., the number of elements in the set.

Following all K trials, an approximation to the probability density $p_j[n]$ for values of $Y[n]$ in $I_j[n]$ is

$$p_j[n] = \frac{f_j[n]}{K(\eta_j[n] - \eta_{j-1}[n])}, \tag{10}$$

and a piecewise linear approximation to the distribution function for $Y[n]$ is defined by the set of straight-line segments joining the points $(\eta_j[n], g_j[n])$, where

$$g_0[n] = 0, \quad g_j[n] = \sum_{\ell=1}^{j} f_\ell[n]/K,\ j = 1, \ldots, d.$$

This approximation to the distribution function can be used as the basis for determining a coverage interval. There is a trade-off, controlled by the number d of intervals and the choice of those intervals, between the amount of information stored and the degree of approximation in the coverage interval obtained from that information. In the case when $d = K$ and each interval endpoint $\eta_j[n]$ is set equal to an output quantity value $y^{(k)}[n]$, the frequencies $f_j[n]$ will all equal one, and the coverage interval obtained will be essentially that obtained from a direct implementation of a Monte Carlo method.

We call the use of updating formulae for the evaluation of measurement uncertainty 'Updating Monte Carlo' (UMC), and give an algorithmic description in Algorithm 3, where *CalcCredUp* in line 21 denotes the calculation of point-wise coverage intervals in terms of the information $\{\eta_j[n], f_j[n]\}$ used to define approximations to the distribution functions for the $Y[n]$, according to equation (10).

Algorithm 3: Updating Monte Carlo (UMC)

1 Initialize: set K as the number of Monte Carlo trials;
2 Initialize: set K_{seq} as a factor of K;
3 Initialize: set intervals $I_j[n],\ j = 1, \ldots, d,\ n = 1, \ldots, N$;
4 Initialize: set $\widehat{\boldsymbol{y}} = \boldsymbol{0}$, $\boldsymbol{U}_{\widehat{\boldsymbol{y}}} = \boldsymbol{0}$, $K_0 = 0$;
5 Initialize: set $f_j[n] = 0,\ j = 1, \ldots, d,\ n = 1, \ldots, N$;
6 **for** $\ell = 1$ **to** K/K_{sec} **do**
7 $\quad \widehat{\boldsymbol{y}}_0 = \widehat{\boldsymbol{y}}$, $\boldsymbol{U}_{\widehat{\boldsymbol{y}}_0} = \boldsymbol{U}_{\widehat{\boldsymbol{y}}}$;
8 $\quad f_{0,j}[n] = f_j[n],\ j = 1, \ldots, d,\ n = 1, \ldots, N$;
9 $\quad$ **for** $k = 1$ **to** K_{seq} **do**
10 $\quad\quad$ draw $\boldsymbol{x}^{(k)}$ from $\mathcal{N}(\boldsymbol{x}, V_\varepsilon)$;
11 $\quad\quad$ draw $\boldsymbol{\mu}^{(k)} = (a_1^{(k)}, \ldots, b_{N_a}^{(k)}, b_0^{(k)}, \ldots, b_{N_b}^{(k)})$ from $p_{\boldsymbol{M}}(\boldsymbol{\mu})$;
12 $\quad\quad$ draw $\boldsymbol{\delta}^{(k)}$ from $\prod_{n=1}^{N} \mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
13 $\quad\quad$ $\boldsymbol{y}^{(k)} = (g_{\boldsymbol{\mu}^{(k)}} * \boldsymbol{x}^{(k)}) + \boldsymbol{\delta}^{(k)}$;
14 $\quad$ **end**
15 $\quad$ update $\widehat{\boldsymbol{y}}$ using expression (7);
16 $\quad$ update $\boldsymbol{U}_{\widehat{\boldsymbol{y}}}$ using expression (8);
17 $\quad$ update $f_j[n],\ n = 1, \ldots, N$, using expression (9);
18 $\quad$ update $K_0 = K_0 + K_{\mathrm{seq}}$;
19 **end**
20 **for** $n = 1$ **to** N **do**
21 $\quad \boldsymbol{I}_{Y[n]} =$ `CalcCredUp`$(\{\eta_j[n], f_j[n] \,|\, j = 1, \ldots, d\})$;
22 **end**

4.3. Sequential implementation

The measurement model (4) and (3) can be applied sequentially in time, since the calculation of a value of the measurand for the present time instant depends on the values of the (calculated) measurand and the (observed) output signal for a small number of previous time instants. This fact can be used to derive a memory-efficient sequential implementation of a Monte Carlo method for the calculation of means, point-wise variances and point-wise coverage intervals.

For each time instant n, expressions (4) and (3) can be considered as a measurement model for the output quantity $Y[n]$ with input quantities $\boldsymbol{M} = (A_1, \ldots, A_{N_a}, B_0, \ldots, B_{N_b})$, $X[n-N_b], \ldots, X[n]$, $\widetilde{Y}[n-N_a], \ldots, \widetilde{Y}[n-1]$ and $\Delta[n]$. Realizations $x^{(k)}[n]$ of $X[n]$ are generated by making random draws $\varepsilon^{(k)}[n]$ according to Algorithm 2 and depending on previous random draws $\varepsilon^{(k)}[n-k]$. The values $\widetilde{y}^{(k)}[n-k]$ are the result of evaluating expression (4) for previous time instants, where $\widetilde{y}[n] = 0$ for $n < 1$. Hence, the initial random draws $x^{(k)}[1]$ propagate sequentially through

realizations of the filter equation (4). We call this method 'Sequential Monte Carlo by Filtering' (SMC–F), and provide an algorithmic description in Algorithm 4, where *CalcCredS* denotes calculation of coverage intervals for $Y[n]$ in terms of the information $\{y^{(k)}[n]\}$; cf. [12]. The outputs of the algorithm are the (point-wise) means $\widehat{y}[n]$, variances $u^2(\widehat{y}[n])$ and coverage intervals $\boldsymbol{I}_{Y[n]}$ for $n = 1, \ldots, N$.

Algorithm 4: Sequential Monte Carlo by Filtering (SMC–F)

1 Initialize: Set K as the number of Monte Carlo trials;
2 **for** $k = 1$ **to** K **do**
3 draw $\boldsymbol{\mu}^{(k)} = (a_1^{(k)}, \ldots, b_{N_b}^{(k)})$ from $p_{\boldsymbol{M}}(\boldsymbol{\mu})$;
4 draw $w^{(k)}[1]$ from $\mathcal{N}(0, \sigma^2)$;
5 draw $\delta^{(k)}$ from $\mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
6 $\varepsilon^{(k)}[1] = w^{(k)}[1]$ and $x^{(k)}[1] = x[1] + \varepsilon^{(k)}[1]$;
7 $\widetilde{y}^{(k)}[1] = b_0^{(k)} x^{(k)}[1]$;
8 $y^{(k)}[1] = \widetilde{y}^{(k)}[1] + \delta^{(k)}$;
9 **end**
10 $\widehat{y}[1] = \frac{1}{K} \sum_{k=1}^{K} y^{(k)}[1]$;
11 $u^2(\widehat{y}[1]) = \frac{1}{K-1} \sum_{k=1}^{K} (y^{(k)}[1] - \widehat{y}[1])^2$;
12 $\boldsymbol{I}_{Y[1]} =$ `CalcCredS(`$\{y^{(k)}[1]\}$`)`;
13 **for** $n = 2$ **to** N **do**
14 **for** $k = 1$ **to** K **do**
15 draw $w^{(k)}[n]$ from $\mathcal{N}(0, \sigma^2)$;
16 draw $\delta^{(k)}$ from $\mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
17 $\varepsilon^{(k)}[n] = \sum_{j=1}^{\min(p,n-1)} \varphi_j \varepsilon^{(k)}[n-j] + \sum_{j=1}^{\min(q,n-1)} \vartheta_j w^{(k)}[n-j] + w^{(k)}[n]$;
18 $x^{(k)}[n] = x[n] + \varepsilon^{(k)}[n]$;
19 $\widetilde{y}^{(k)}[n] = \sum_{j=0}^{\min(N_b,n-1)} b_j^{(k)} x^{(k)}[n-j] - \sum_{j=1}^{\min(N_a,n-1)} a_j^{(k)} \widetilde{y}^{(k)}[n-j]$;
20 $y^{(k)}[n] = \widetilde{y}^{(k)}[n] + \delta^{(k)}$;
21 **end**
22 $\widehat{y}[n] = \frac{1}{K} \sum_{k=1}^{K} y^{(k)}[n]$;
23 $u^2(\widehat{y}[n]) = \frac{1}{K-1} \sum_{k=1}^{K} (y^{(k)}[n] - \widehat{y}[n])^2$;
24 $\boldsymbol{I}_{Y[n]} =$ `CalcCredS(`$\{y^{(k)}[n],\ k = 1, \ldots, K\}$`)`;
25 **end**

In a MATLAB implementation of Algorithm 4 the calculations in the inner for-loop (lines 15 to 20) can be replaced by matrix-vector operations, which results in a much smaller computation time:

```
w(1:K,n)       = sigma*randn(K,1);
delta(1:K)     = 2*deltabar*(rand(K,1) - 0.5);
eps(1:K,n)     = dot(phi(1:K,1:p), eps(1:K,n-1:-1:n-p)) + ...
                     dot(theta(1:K,1:q), w(1:K,n-1:-1:n-q)) + w(1:K,n);
X(1:K,n)       = x(n) + eps(1:K,n);
Ytilde(1:K,n)  = sum(B(1:K,1:Nb+1).*X(1:K,n:-1:n-Nb), 2) - ...
                    sum(A(1:K,1:Na).*Ytilde(1:K,n-1:-1:n-Na), 2);
Y(1:K,n)       = Ytilde(1:K,n) + delta(1:K);
```

where ".*" denotes element-wise multiplication. Furthermore, it is only necessary to store columns $n-q$ to n of the matrix `w`, columns $n-p$ to n of `eps`, columns $n-Nb$ to n of `X`, and columns $n-Na$ to $n-1$ of `Ytilde`. Therefore, for example, the above calculation of `Ytilde` can be replaced by

```
Ytilde(1:K,1:Na+Nb+1) = [(sum(B(1:K,1:Nb+1).*X(1:K,1:Nb+1),2) - ...
      sum(A(1:K,1:Na).*Ytilde(1:K,1:Na),2), Ytilde(1:K,1:Na+Nb)];
```

An alternative representation of the measurement model (4) and (3) is obtained by a transformation to the state-space form [7, 24]:

$$\boldsymbol{z}^\top[n+1] = \begin{pmatrix} -A_1 & \cdots & \cdots & -A_{N_a} \\ 0 & & & \\ \vdots & & \boldsymbol{I}_{N_a-1} & \\ 0 & & & \end{pmatrix} \boldsymbol{z}^\top[n] + \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} X[n], \tag{11}$$

$$Y[n] = (D_1, \ldots, D_{N_a})\boldsymbol{z}^\top[n] + B_0 X[n] + \Delta[n], \tag{12}$$

where $D_j = B_j - B_0 A_j$, $\boldsymbol{I}_{N_a-1}$ is the identity matrix of dimension $N_a - 1$, and it is assumed that $\boldsymbol{B} = (B_1, \ldots, B_{N_b})^\top$ and $\boldsymbol{A} = (A_1, \ldots, A_{N_a})^\top$ are padded with zeros to have the same length, i.e., $N_a = N_b$. We call this method 'Sequential Monte Carlo for State-Space models' (SMC–SS) and give an algorithmic description in Algorithm 5, where it is assumed that $N_a = N_b$. Note that different notations for the state-space form are possible [24]. However, Algorithm 5 implements the form given in equation (11). For a different notation of the state-space form, lines 20-22 in Algorithm 5 have to adapted accordingly.

Algorithm 5: Sequential Monte Carlo for State-Space models (SMC–SS)

Initialize: Set K as the number of Monte Carlo trials;
for $k = 1$ **to** K **do**
  draw $\boldsymbol{\mu}^{(k)} = (a_1^{(k)}, \ldots, b_{N_a}^{(k)})$ from $p_{\boldsymbol{M}}(\boldsymbol{\mu})$;
  $d_j^{(k)} = b_j^{(k)} - b_0^{(k)} a_j^{(k)}, \quad j = 1, \ldots, N_a$;
  draw $w^{(k)}[1]$ from $\mathcal{N}(0, \sigma^2)$;
  draw $\delta^{(k)}$ from $\mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
  $\varepsilon^{(k)}[1] = w^{(k)}[1]$ and $x^{(k)} = x[1] + \varepsilon^{(k)}[1]$;
  $y^{(k)}[1] = b_0^{(k)} x^{(k)} + \delta^{(k)}$;
  $z_j^{(k)} = 0, \quad j = 1, \ldots, N_a$;
end
$\widehat{y}[1] = \frac{1}{K} \sum_{k=1}^{K} y^{(k)}[1]$;
$u^2(\widehat{y}[1]) = \frac{1}{K-1} \sum_{k=1}^{K} (y^{(k)}[1] - \widehat{y}[1])^2$;
$\boldsymbol{I}_{Y[1]} =$ `CalcCredS(`$\{y^{(k)}[1]\}$`)`;
for $n = 2$ **to** N **do**
  for $k = 1$ **to** K **do**
    draw $w^{(k)}[n]$ from $\mathcal{N}(0, \sigma^2)$;
    draw $\delta^{(k)}$ from $\mathcal{U}(-\overline{\Delta}, \overline{\Delta})$;
    $\varepsilon^{(k)}[n] = \sum_{j=1}^{\min(p,n-1)} \varphi_j \varepsilon^{(k)}[n-j] + \sum_{j=1}^{\min(q,n-1)} \vartheta_j w^{(k)}[n-j] + w^{(k)}[n]$;
    $x^{(k)} = x[n] + \varepsilon^{(k)}[n]$;
    $y^{(k)}[n] = \boldsymbol{d}^{(k)} (\boldsymbol{z}^{(k)})^\top + b_0^{(k)} x^{(k)} + \delta^{(k)}$;
    $z_1^{(k)} = -\boldsymbol{a}^{(k)} (\boldsymbol{z}^{(k)})^\top + (1, 0, \ldots, 0)^\top x^{(k)}$;
    $z_j^{(k)} = z_{j-1}^{(k)}, \quad j = 2, \ldots, N_a - 1$;
  end
  $\widehat{y}[n] = \frac{1}{K} \sum_{k=1}^{K} y^{(k)}[n]$;
  $u^2(\widehat{y}[n]) = \frac{1}{K-1} \sum_{k=1}^{K} (y^{(k)}[n] - \widehat{y}[n])^2$;
  $\boldsymbol{I}_{Y[n]} =$ `CalcCredS(`$\{y^{(k)}[n],\ k = 1, \ldots, K\}$`)`;
end

The state-space representation of model (4) and (3) does not require the values $\widetilde{y}^{(k)}[n-j]$ for $j > 0$ to be stored. Instead, the state vector $\boldsymbol{z}$ encodes filter results of previous time instants. Similarly to Algorithm 4, in a MATLAB implementation the calculations in the inner for-loop of Algorithm 5 can be replaced by matrix-vector operations, and do not require all the columns of the matrices to be stored:

```
w(1:K,n)     = sigma*randn(K,1);
delta(1:K)   = 2*deltabar*(rand(K,1) - 0.5);
eps(1:K,n)   = dot(phi(1:K,1:p), eps(1:K,n-1:-1:n-p)) + ...
                   dot(theta(1:K,1:q), w(1:K,n-1:-1:n-q)) + w(1:K,n);
X(1:K)       = x(n) + eps(1:K,n);
Y(1:K)       = sum(D(1:K,1:Na).*Z(1:K,1:Na),2) + b0(1:K).*X(1:K) + ...
                   delta(1:K);
Z(1:K,1:Na) = [(-sum(A(1:K,1:Na).*Z(1:K,1:Na),2) + X(1:K)), Z(1:K,1:Na-1)];
```

4.4. Comparison

The algorithms differ mainly in the amount of computer memory that they require. Let K denote the number of Monte Carlo trials, N the signal length and $M = N_a + N_b + 1$ the number of filter coefficients. For ease of presentation we assume for the subsequent storage computations that the observational noise is independent and identically distributed Gaussian ($p = q = 0$), and the storage of input parameters (here, $\boldsymbol{x}$, σ, $\overline{\Delta}$ and K, which are common to all the algorithms) and output parameters (mean, variances and coverage intervals) is not counted. Then, Algorithm 1 (BMC) requires to store N filter input values (line 3), M filter coefficients (line 4), N values for the systematic correction (line 5) and KN output quantity values (line 6), giving a total of $N(K + 2) + M$ array elements. Note that the N filter input values and M filter coefficients can be overwritten in each Monte Carlo iteration. Thus, there is no need to store the KN filter input values and KM filter coefficients used in the algorithm. Similarly, for Algorithm 3 (UMC) the number of array elements stored is given by this same expression but with K replaced by K_{seq}, and it is also necessary to store the mean vector (N elements), covariance matrix (N^2 elements) and point-wise histograms ($2dN$ elements) that are updated after each set of K_{seq} trials (lines 15 to 17). The sequential algorithms 4 and 5 require to store K realisations of the M filter coefficients, K realisations of the noise values $\varepsilon^{(k)}[n]$ and systematic correction values δ and the obtained K realisations of the output value $y^k[n]$. For the algorithm 4 (SMC–F), additionally K realisations of the N_b values $x^{(k)}[n-1], \ldots, x^{(k)}[n-N_b]$ and the N_a+1 values $\widetilde{y}^{(k)}[n], \ldots, \widetilde{y}^{(k)}[n-N_a]$ are required. For the state-space based algorithm 5 (SMC–SS) instead the N_a-dimensional state-space vectors $\boldsymbol{z}^{(k)}[n]$ and N_b-dimensional observation equation vectors $\boldsymbol{d}^{(k)}$ are employed, which requires KN_a and KN_b array elements to be stored, respectively. Table 1 summarizes the numbers of stored array elements counted in this way for the four algorithms BMC, UMC, SMC–F and SMC–SS.

Table 1. Storage requirement for the case that point-wise statistics are considered, with K the number of Monte Carlo trials, N the signal length and M the number of filter coefficients. The noise process $\varepsilon[n]$ is assumed to be independent and identically distributed Gaussian. The third column contains approximate values for the storage requirement for the typical example described in the text.

Algorithm	Storage	Example
BMC (direct)	$N(K+2)+M$	2.0×10^8
UMC (with correlations)	$N(N+K_{\text{seq}}+2d+3)+M$	6.4×10^6
UMC (no correlations)	$N(K_{\text{seq}}+2d+4)+M$	2.4×10^6
SMC–F (filter)	$K(2M+3)$	2.3×10^6
SMC–SS (state-space)	$K(2M+2)$	2.2×10^6

The storage requirement for BMC is dominated by the term KN, which corresponds to the set of output values $\{y^{(k)}[n] : k = 1, \ldots, K,\ n = 1, \ldots, N\}$. Typically, the set

of output quantity values is stored in a matrix of dimension $K \times N$, since this allows for the convenient calculation of mean values, covariances, coverage intervals and other statistics. The size of this matrix restricts the maximum number of Monte Carlo trials as many computer programs store matrices in contiguous elements in computer memory. Depending on the available free memory and type of operating system (i.e., 32 bit or 64 bit) this restricts the maximum possible array size. The restriction is important, because a large number of trials may be needed in order to achieve a prescribed numerical accuracy in the results (see Section 5). For any procedure, an almost arbitrarily large number of trials can be achieved by writing the batches $\boldsymbol{y}^{(k)}$ to hard disk. However, such an approach results in a much longer run time and a rather inconvenient calculation of, e.g., coverage intervals.

When the product KN is much larger than $K_{\mathrm{seq}}N$, N^2 and $2dN$, which is usually the case, the results in Table 1 show that the updating procedure (UMC) is clearly superior to a direct implementation of a Monte Carlo method in terms of its storage requirement. In particular, the storage requirement for the UMC procedure does not depend on the chosen number of Monte Carlo trials K. Furthermore, if only point-wise results are required, the storage of the covariance matrix (N^2 elements) that is updated after each set of K_{seq} trials can be replaced by that of N variances, and then the storage requirement of UMC is dominated by $K_{\mathrm{seq}}N$ or $2dN$.

Similarly, since the product KN is usually much larger than KM, the results show that the sequential procedures (SMC–F and SMC–SS) are also clearly superior to a direct implementation. The sequential procedures allow for a much larger number of trials than a direct implementation (BMC), since their storage requirements do not depend on the signal length N.

To illustrate the superiority of the alternative procedures in terms of storage requirement we consider a typical example. Assume a number of Monte Carlo trials of $K = 10^5$, a signal length of $N = 2 \times 10^3$, a number of filter coefficients of $M = 10$, and for the method UMC $d = 100$ and $K_{\mathrm{seq}} = 10^3$. Then (cf. Table 1), the number of array elements to be stored for the individual procedures is approximately 2.0×10^8 (BMC), 6.4×10^6 (UMC with correlations), 2.4×10^6 (UMC without correlations), 2.3×10^6 (SMC–F) and 2.2×10^6 (SMC–SS).

It is worth noting that the number of elementary operations (ElOps) for the calculation of the values $y^{(k)}[n]$ is the same, namely $KN(4M + 3)$, for all procedures. However, the calculations of means, covariances and coverage intervals using the updating formulae (7), (8) and (9) require more ElOps than for the other procedures because it is necessary to evaluate intermediate results. Overall, the UMC method provides a trade-off between the memory-efficiency of the sequential procedures and the availability of the (complete) joint PDF. The drawback of the procedure is that coverage intervals are determined less well because of the fixed number of frequency bins and the partition of $\mathbb{R}$ into intervals to be defined in a preliminary stage of the procedure (see Section 4.2).

In general, the complete set of values of the measurand (as provided by the

BMC procedure) is required to account for non-linear dependencies between estimates at different time instants, since these dependencies are not fully captured by the covariance matrix. These dependencies are important especially when an operator or a functional is applied to the dynamic measurand $\boldsymbol{Y}$. For the UMC method operators or functionals on the discrete-time estimate can easily be taken into account by including the corresponding evaluation in the for-loop of Algorithm 3 directly after line 13.

For the sequential methods SMC–F and SMC–SS functionals and operators can be taken into account, provided that they can be evaluated sequentially. For example, an approximate calculation of the integral of the measurand using the trapezoidal rule

$$\int_0^{(N-1)T_s} y(t)\mathrm{d}t \approx \frac{T_s}{2}\sum_{n=1}^{N-1}(Y[n]+Y[n+1]) \equiv F, \tag{13}$$

where T_s is the sampling interval, can be carried out sequentially as follows:

(i) Set $F^{(k)} = 0$ for $k = 1, \ldots, K$;

(ii) Calculate sequentially, starting from $n = 2$:

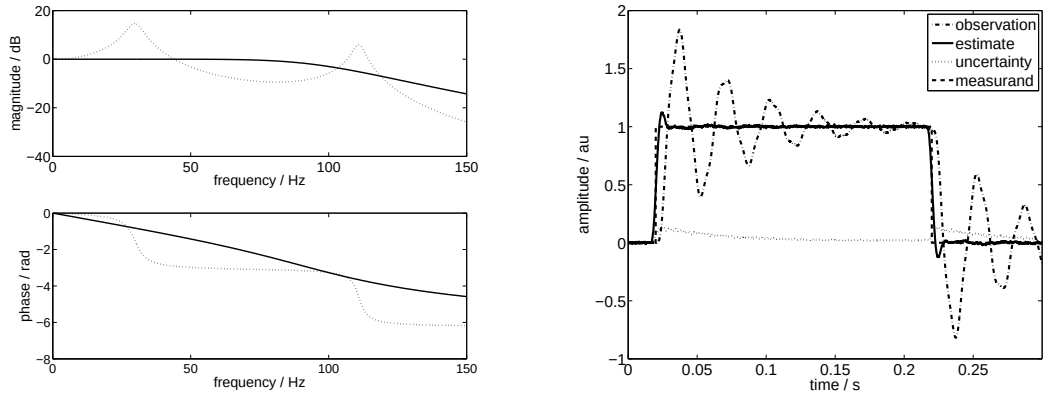
$$F^{(k)} := F^{(k)} + \frac{T_s}{2}(y^{(k)}[n] + y^{(k)}[n-1]).$$

The resulting set of values $F^{(k)}$, $k = 1, \ldots, K$, of the (approximate) integral F can be used to calculate an estimate of F, the associated standard uncertainty and a coverage interval for F. In this way, correlations and non-linear dependencies between estimates at different time instants are taken into account.

According to the Riesz representation theorem all continuous linear functionals on the space of square-integrable functions can be written in terms of an integral

$$\phi(y) = \int f(t)y(t)\,\mathrm{d}t,$$

where the function $f(t)$ is uniquely determined by ϕ. Hence, the sequential approaches allow for an evaluation of all continuous linear functionals and integral operators acting on the dynamic measurand. However, the functional must be known at the time the analysis considered above is carried out. Thus, if the functional is not known at the time the measurement of $x(t)$ is being made one may have to store the information which serve as input to the sequential algorithms. More precisely, in this case information about the PDFs $p_{\boldsymbol{M}}(\boldsymbol{\mu})$ (e.g., SMC-SS line 3), $\mathcal{N}(0, \sigma^2)$ (e.g., SMC-SS line 5) and $\mathcal{U}(-\overline{\Delta}, \overline{\Delta})$ (e.g., SMC-SS line 6) has to be stored together with the observed sensor output signal $\mathbf{x}$. Note that the same holds true also for UMC (with correlations) as the covariance matrix encodes only linear dependencies between different time instants. The application of BMC allows the storage of the calculated set of output quantity values $\{y^{(k)}[n] \mid n = 1, \ldots, N,\ k = 1, \ldots, K\}$ as a characterisation of the full PDF that encodes degree of belief about the measurand $\mathbf{y}$.



(a) (b)

Figure 1. (a) Frequency response of measuring system (dotted) and its product with the deconvolution filter (solid). (b) Simulated system output (dotted), estimated input (solid) and its associated uncertainty (dash-dotted). These results are essentially the same for all procedures.

5. Example

We illustrate the application of the proposed procedures by means of a simulated example. We assume observations are obtained from a measuring system that can be modelled by a cascade of two damped mass-spring systems:

$$H(s) = H_1(s)H_2(s), \qquad H_i(s) = \frac{S_0(2\pi f_{0,i})^2}{s^2 + 2\delta_i(2\pi f_{0,i})s + (2\pi f_{0,i})^2}, \tag{14}$$

where the system parameters and their associated uncertainties are $S_0 = 1.000\ 0$ (0.009 5), $\delta_1 = 0.100$ (0.063), $f_{0,1} = 30.0$ Hz (1.9 Hz), $\delta_2 = 0.020$ (0.019) and $f_{0,2} = 111.0$ Hz (7.0 Hz). Due to physical reasons all parameters are known to be non-negative. Such dynamic systems occur, for instance, in force measurement [25].

The chosen sampling frequency is $f_s = 10$ kHz, which allows for a discretisation of the continuous model (14) using the impulse invariance technique [17] with negligible error. The resulting discrete system model is minimum-phase [17]. For a deconvolution filter we apply the reciprocal discrete system in cascade with an order 4 Butterworth low-pass filter with cut-off frequency at 100 Hz for noise attenuation; see Fig. 1(a). The time sample delay of the deconvolution filter is approximately 44 time samples.

For the measurand we consider a rectangular shaped signal that serves as the input signal to the (discretized) measuring system model; see Fig. 1(b). The resulting maximum regularization error is 3.6×10^{-4} and can thus be neglected compared with other uncertainty contributions. As observational noise we consider independent and identically distributed Gaussian noise $\varepsilon[n] \sim \mathcal{N}(0, \sigma^2)$ with variance $\sigma^2 = 10^{-6}$.

For the state-of-knowledge PDF for the parameters of model (14) we assume a (one-side) truncated multivariate normal distribution with mean given by the parameter estimates and diagonal covariance matrix. The truncation point is chosen at zero for

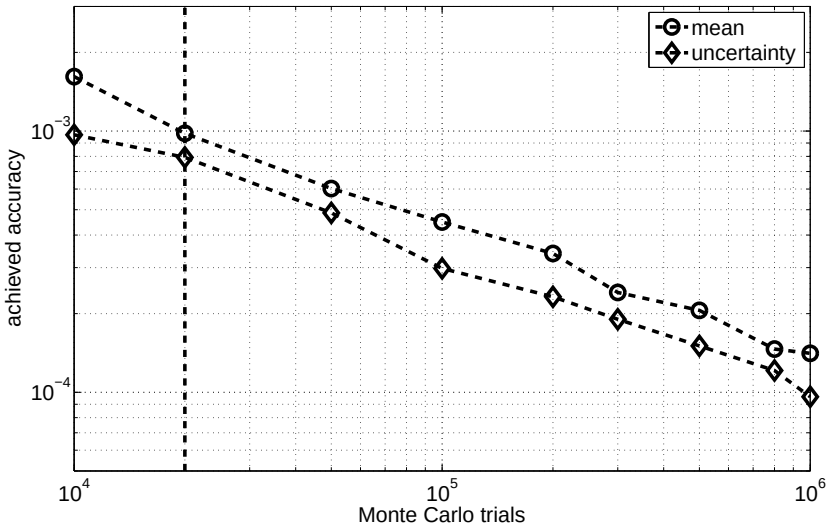


Figure 2. Numerical accuracy of Monte Carlo results (estimate and associated standard uncertainty) for different numbers of Monte Carlo trials. The vertical line represents the maximum possible number of trials for the BMC procedure for the example considered on a standard personal computer with a 32-bit operating system using MATLAB.

the marginal distributions to account for the non-negativity of the physical parameters. Estimates of the measurand $\boldsymbol{Y}$ and the associated uncertainties were obtained using Algorithm 5 with $K = 10^6$ Monte Carlo trials, and are shown in Fig. 1(b).

The Monte Carlo results are not exact, and will vary each time the Monte Carlo calculation is applied. In order to obtain a prescribed numerical accuracy (with high probability) in the results, the number of trials has to be chosen sufficiently large. To illustrate the randomness of the results and the effect of the choice of number of trials we carried out the calculation for different numbers of trials repeatedly with different seeds for the (pseudo-)random number generators used. For each number of trials, Algorithm 5 was applied 100 times, resulting in estimates $(\widehat{y}_k[1], \ldots, \widehat{y}_k[N])$ and associated standard uncertainties $(u(\widehat{y}_k[1]), \ldots, u(\widehat{y}_k[N]))$ for $k = 1, \ldots, 100$. For each time instant n, the standard deviations $r_y[n]$ of $\{\widehat{y}_k[n] : k = 1, \ldots, 100\}$ and $r_{u(y)}[n]$ of $\{u(\widehat{y}_k[n]) : k = 1, \ldots, 100\}$ are calculated. The maximum values $\max_n\{r_y[n]\}$ and $\max_n\{r_{u(y)}[n]\}$ for the different numbers of Monte Carlo trials are shown in Fig. 2. As can be seen from Fig. 2, in order to achieve a numerical accuracy of more than two decimal digits with high probability, at least 10^5 trials are necessary for this example.

For the application of the BMC procedure to this example the maximum possible number of trials is about 2×10^4 on a standard personal computer with a 32-bit operating system using MATLAB. Hence, the maximum achievable numerical accuracy (with high probability) with the BMC procedure is two decimal digits for this example.

We also encountered the *Out of memory* message in MATLAB for a signal length of $N = 3\ 000$ samples and 1.2×10^6 trials on a workstation computer with 24 cores,

96 GB of RAM running a 64-bit Linux operating system. With the sequential methods SMCP–F and SMCP–SS the same calculations run on a desktop computer without difficulty.

As mentioned in Sections 4.2 and 4.4, there is a trade-off in the updating procedure (UMC) between storage requirement and the numerical accuracy of coverage intervals. This trade-off is controlled by the number b of intervals (frequency bins) chosen. For $b = K$ the storage requirement for UMC is essentially equal to that for the application of the direct implementation (BMC). As b decreases, the approximation error in the calculation of coverage intervals increases. For the example considered here the use of $b = 100$ intervals resulted in a maximum error in the calculated coverage interval of about 8 % relative to the length of the interval returned by BMC. As the storage requirement of the UMC procedure does not depend on the number K of Monte Carlo trials, the achievable numerical accuracy is restricted only by the numerical stability of the updating formulae and the interpolation error for the calculation of coverage intervals.

6. Conclusions

The measurement models arising in dynamic measurement problems are generally non-linear. For models with finite impulse response (FIR) filters exact formulae for the calculation of mean and variance are available. Models with infinite impulse response (IIR) filters generally require linearization to apply closed formulae for the calculation of mean and variance. Hence, the calculation of measurement uncertainty may then require the application of a Monte Carlo method because the magnitudes of the errors incurred by linearising the measurement models can be appreciable. Moreover, the calculation of coverage intervals generally requires the application of a Monte Carlo method unless distributional assumptions can be made.

However, a direct implementation of a Monte Carlo method can be intractable on many computers since the storage requirement to achieve a prescribed numerical accuracy in the results can be large compared with the available computer memory. Two memory-efficient alternatives to the direct implementation have therefore been described, which permit a reliable evaluation of measurement uncertainty.

One approach is based on applying updating formulae for calculating means, covariances and point-wise histograms. The storage requirement for the resulting procedure is independent of the total number of Monte Carlo trials, but it delivers point-wise coverage intervals that are determined less well, and requires a greater number of elementary floating-point operations to deliver results compared to other approaches.

A second approach is based on evaluating the measurement model sequentially using a representation of the measurement model either as a discrete filter or in state-space form. The resulting procedures require that the output quantity values for a small number of time instants only are stored, deliver results that are identical to those obtained from a direct implementation, and require the same number of elementary

floating-point operations as the direct implementation.

Acknowledgments

The National Measurement Office of the UK Department for Business, Innovation and Skills supported the contribution of the National Physical Laboratory to this work as part of its Mathematics and Modelling for Metrology programme. We thank our colleagues Trevor Esward and Maurice Cox at NPL for their comments on earlier drafts of the paper.

References

[1] A. Link, A. Täubner, W. Wabinski, T. Bruns, and C. Elster. Modelling accelerometers for transient signals using calibration measurements upon sinusoidal excitation. *Measurement*, 40:928–935, 2007.

[2] C. Elster, A. Link, and T. Bruns. Analysis of dynamic measurements and determination of time-dependent measurement uncertainty using second-order models. *Measurement Science and Technology*, 18:3682–3687, 2007.

[3] A. Link, B. Glöckner, C. Schlegel, R. Kumme, and C. Elster. System identification of force transducer for dynamic measurements. In *Proceedings of XIX IMEKO World Congress on Fundamental and Applied Metrology*, 2009.

[4] T. S. Clement, P. D. Hale, D. F. Williams, C. M. Wang, and A. Dienstfrey. Calibration of sampling oscilloscopes with high-speed photodiodes. *IEEE Trans. Microwave Theory and Techn.*, 54:3173–3181, 2006.

[5] M. Bieler, M. Spitzer, K. Pierz, and U. Siegner. Improved Optoelectronic Technique for the Time-Domain Characterization of Sampling Oscilloscopes. *CPEM Special Issue of Trans. I & M*, 1, 2008.

[6] P. D. Hale, A. Dienstfrey, J. C. M. Wang, D. F. Williams, A. Lewandowski, D. A. Keenan, and T. S. Clement. Traceable waveform calibration with a covariance-based uncertainty analysis. *IEEE Trans. Intrum. Meas.*, 58:3554–3568, 2009.

[7] A. Link and C. Elster. Uncertainty evaluation for IIR (infinite impulse response) filtering using a state-space approach. *Measurement Science and Technology*, 20, 2009.

[8] C. Elster, S. Eichstädt, and A. Link. Uncertainty evaluation of dynamic measurements in line with gum. In *XIX IMEKO World Congress on Fundamental and Applied Metrology*, 2009.

[9] S. Eichstädt, A. Link, T. Bruns, and C. Elster. On-line dynamic error compensation of accelerometers by uncertainty-optimal filtering. *Measurement*, 43:708–713, 2010.

[10] J. P. Hessling. A novel method of dynamic correction in the time domain. *Meas. Sci. Technol.*, 19:10pp, 2008.

[11] BIPM, IEC, IFCC, ISO, IUPAC, IUPAP, and OIML. *Guide to the Expression of Uncertainty in Measurement*. International Organization for Standardization, Geneva Switzerland, 1995.

[12] BIPM, IEC, IFCC, ISO, IUPAC, IUPAP, and OIML. *Evaluation of Measurement Data - Supplement 1 to the 'Guide to the Expression of Uncertainty in Measurement' - Propagation of Distributions Using a Monte Carlo Method*. Joint Commitee for Guides in Metrology, Bureau International des Poids et Mesures, JCGM 101, 2008.

[13] BIPM, IEC, IFCC, ISO, IUPAC, IUPAP, and OIML. *Evaluation of Measurement Data - Supplement 2 to the 'Guide to the Expression of Uncertainty in Measurement' - Extension to any number of output quantities*. Joint Commitee for Guides in Metrology, Bureau International des Poids et Mesures, JCGM 102, 2011.

[14] S. Eichstädt, C. Elster, T. J. Esward, and J. P. Hessling. Deconvolution filters for the analysis of dynamic measurement processes: a tutorial. *Metrologia*, 47:522–533, 2010.

[15] J. P. Hessling. A novel method of estimating dynamic measurement errors. *Meas. Sci. Technol.*, 17:2740–2750, 2006.

[16] C. Elster and A. Link. Uncertainty evaluation for dynamic measurements modelled by a linear time-invariant system. *Metrologia*, 45:464–473, 2008.

[17] A. V. Oppenheim and R. W. Schafer. *Discrete-Time Signal Processing*. Prentice Hall, 1989.

[18] S. M. Riad. The deconvolution problem: an overview. *IEEE proceedings*, 74:82–85, 1988.

[19] R. Pintelon, Y. Rolain, M. Vanden-Bosche, and J. Schoukens. Towards an ideal data acquisition channel. *IEEE Trans. Intrum. Meas.*, 39:116–120, 1990.

[20] L. Ljung. *System Identification: Theory for the User*. Prentice Hall, 1998.

[21] M. B. Priestley. *Spectral Analysis and Time Series*. Academic Press, New York, 1996.

[22] Uwe Hassler. *Stochastische Integration und Zeitreihenmodellierung*. Springer, Berlin, 2007.

[23] G. Wübbeler, P. M. Harris, M. G. Cox, and C. Elster. A two-stage procedure for determing the number of trials in the application of a Monte Carlo method for uncertainty evaluation. *Metrologia*, 47:317–324, 2010.

[24] John S. Bay. *Fundamentals of linear state space systems*. WCB/McGraw-Hill, Boston [u.a.], 1999.

[25] A. L. Shestakov. Dynamic error correction method. *IEEE Trans. Instrum. Meas.*, 45:250–255, 1996.